

LLM-Based Semantic Chunking for Retrieval-Augmented Generation

A Comparison with Rule-Based Chunking Strategies

Bachelor's Thesis

Ivan Antunovic

Supervisor: Dr. Marian Lux

University of Vienna

September 30, 2026

Abstract

The use of large language models (LLMs) is more widespread than ever, and with their integration into more and more industries a further rise seems inevitable. However, these models are trained on past data and generate responses based on probabilities, which can lead to hallucinations. Retrieval-augmented generation (RAG) counteracts this problem by supplying the model with relevant documents. To do so, a RAG system has to split the documents into chunks, a pre-processing step that determines what can be retrieved and thereby the accuracy of the answers. In this thesis, an LLM-based chunker is implemented as an alternative to rule-based chunking strategies and published as a Python library on PyPI. It uses a small local model that reads the document sentence by sentence and decides where the topic changes; optional post-processing steps are meant to improve retrieval further. The chunker is evaluated on three documents of different structure (a technical standard, a PDF handbook and narrative prose) against five baselines tuned to the same mean chunk length: four splitters from LangChain and the same chunker without the model. The statistical analysis uses paired McNemar tests with a Holm correction over 30 comparisons, with 400 to 549 questions per document. In six of the 30 comparisons, the LLM chunker retrieves significantly more answers than the splitters that cut sentences apart, but it is not significantly better than the splitters that keep sentences intact or than the same chunker without the model. This indicates that its advantage comes from respecting sentence boundaries rather than from the model's topic decisions. Heading detection and topic labels did not improve retrieval significantly, and the filter for chunks without substantive content even lowered it. In practice, a length-matched recursive splitter that prefers sentence ends reaches comparable hit rates in a few seconds, against a mean of about three hours per document on an 8 GB MacBook Air.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem statement and research question	4
2	Background and Related Work	5
2.1	Chunking in retrieval-augmented generation	5
2.2	Rule-based and embedding-based strategies	5
2.3	LLM-based chunking strategies	6
2.4	Shortcomings in current literature and methodological commitments	7
3	Implementation	8
3.1	Overview and design goals	8
3.2	Architecture	9
3.2.1	Incremental boundary detection	9
3.2.2	Size cap and split policy	10
3.2.3	Heading awareness	10
3.2.4	Post-processing	10
3.2.5	Configuration and the LLM client	11
3.3	Developer guide	11
3.3.1	Installation	11
3.3.2	The native API and the framework adapters	12
3.3.3	Extension points	13
3.4	Verification	13
3.4.1	Determinism	13
3.4.2	The question sets	13
3.5	The evaluation harness	14
3.6	Use of AI tools	14
3.7	Availability	14
4	Evaluation	15
4.1	Datasets	15
4.1.1	Question sets	15
4.2	Experimental design	16
4.2.1	Compared strategies	16

4.2.2	Retrieval and hit criterion	17
4.2.3	Statistical analysis	17
4.3	Results	18
4.3.1	Retrieval accuracy	18
4.3.2	Paired comparisons	18
4.4	The low-information filter	18
4.5	Where the differences come from	19
4.6	Ablations of the LLM arm	20
4.7	Parent-child retrieval	21
4.8	Cost	21
4.9	Robustness and threats to validity	22
4.9.1	Robustness	22
4.9.2	Threats to validity	22
5	Findings and Lessons Learned	24
5.1	Summary of findings	24
5.2	Lessons learned	25
5.2.1	A fair comparison needs more than equal chunk length	25
5.2.2	Changes to the plan will happen	25
5.2.3	Cached results ensure clean tracking	25
5.2.4	Working with limited hardware	25
5.3	Limitations and future work	25
	Bibliography	27

Chapter 1

Introduction

1.1 Motivation

Large language models (LLMs) demonstrate high accuracy by storing factual knowledge in their parameters, yet they remain prone to hallucinations, and their ability to access and update knowledge is limited [7]. Retrieval-augmented generation (RAG) addresses these shortcomings by combining the parametric language generator of an LLM with a non-parametric external document index, from which relevant information is retrieved to generate more accurate answers [7]. Within the RAG pipeline, document chunking is the pre-processing step that splits the text into units and thereby defines the search granularity of the vector index [13].

Many chunking variants exist for this step, but they differ in accuracy and are not suitable for every document type or domain. Recent research introduced a newer mechanism, LLM-based chunking, which dynamically identifies narrative topic shifts to improve the quality of each vector embedding [3]. However, LLM-driven segmentation introduces a considerable computational latency compared to the existing baselines, as the experiments with LumberChunker showed [3].

Furthermore, empirical evidence suggests that on real-world documents, sentence-level positional proximity already provides a strong natural heuristic for semantic shifts, so that the computational overhead is not justified by the gain [11]. This motivates an evaluation of whether LLM-based chunking brings a real advantage over length-matched baselines, or whether preserving the integrity of sentences is already sufficient for effective retrieval.

1.2 Problem statement and research question

The research question is: *Does a local language model place better chunk boundaries than rule-based chunking techniques, and under which conditions?* Secondary questions concern the effect of the optional features of the library: heading detection, the low-information filter, topic enrichment and the choice of the split point when a chunk exceeds the size cap.

Chapter 2

Background and Related Work

2.1 Chunking in retrieval-augmented generation

Standard large language models (LLMs) struggle with knowledge-intensive tasks because their internal knowledge is limited by a static training cut-off and their output lacks verifiable sources [7]. Retrieval-augmented generation (RAG) resolves these limitations by coupling pre-trained parametric generators (such as BART) with non-parametric memory in the form of an external dense vector index [7]. In a RAG pipeline, raw source documents are first split into retrievable text units, the so-called *chunks* [13], which are then converted into dense vector representations by a neural encoder and finally searched through maximum inner product search (MIPS) or approximate nearest-neighbour algorithms [7].

This segmentation is a pre-processing step that determines the quality of the subsequent vector index [13]. Downstream components, such as cross-encoder rerankers, have to rely on the initial segmentation; errors made during this step cannot be repaired later, and the subsequent components are forced to process chunks that are either truncated or contextually degraded [3].

Chunking can fail in two ways. Chunks that are too small or cut mid-sentence lack sufficient factual context, which leads to retrieval failures and a higher risk of hallucinations by the LLM. Conversely, chunks that are too large contain irrelevant content, which dilutes the vector embedding and creates a semantic misalignment with the query vector during dense retrieval [13].

Despite its decisive role in a RAG pipeline, chunking has historically been treated as an arbitrary pre-processing step done through fixed character cuts rather than as an architectural variable [11]. For comparative studies, recent literature emphasises the need to isolate chunking from the rest of the RAG pipeline in order to guarantee consistency across the compared splitting variants [13].

2.2 Rule-based and embedding-based strategies

Fixed-size chunking splits a source document into uniform segments based on a predetermined character or token limit [11]. Although the procedure is computationally simple and deterministic, it disregards syntax and thematic organisation [13]. This leads to chunks that are cut mid-topic and thus to a potential degradation in retrieval quality [11]. To mitigate these effects, RAG implementations let consecutive fixed-size segments overlap in a sliding window, typically by

10 % to 20 % of the chunk length [13]. However, a recent empirical evaluation found that overlap provides no measurable improvement in answer quality and only increases the indexed text and the memory overhead [1].

In contrast, *recursive character splitting* traverses the document through an ordered hierarchy of typographical separators and thereby preserves its structure where possible [3]. Typically, such splitters cascade through paragraph breaks, line breaks, sentence punctuation and finally spaces: when a segment still exceeds the size limit, the algorithm moves one step down the hierarchy to find a split point [3].

Embedding-based strategies differ from these typographical heuristics in that they detect boundaries semantically. *Semantic chunking* converts individual sentences or passages into dense vector embeddings, numerical representations in a high-dimensional space, using models such as Sentence-BERT (SBERT) [12]. The cosine distance between consecutive segments then determines whether a thematic shift has occurred. More advanced approaches such as Max-Min semantic chunking compare the similarity of each incoming sentence to the current chunk with the minimum similarity within that chunk, so that the text is split without a fixed window size [6], [13]. However, the benefit of semantic chunking depends strongly on the data. On artificially stitched datasets with high topic diversity, it preserves topic boundaries better than recursive algorithms, but on real-world documents, which already provide natural cues for topic boundaries, the retrieval accuracy does not differ substantially while the computational cost is higher [11]. The additional cost, latency and memory needed to compute sentence-level embeddings are therefore rarely justified for standard structured text [11].

2.3 LLM-based chunking strategies

LLM-based segmentation strategies replace distance thresholds with generative models that identify semantic shifts in a document [13]. Earlier work in this field includes LumberChunker, which divides a text into paragraphs and assigns them sequential IDs starting at 1. Consecutive paragraphs are collected into a group until a target token count is exceeded, and the group is then submitted to an LLM with a prompt that asks for the ID of the paragraph at which the narrative or topical focus changes [3].

On a benchmark of 3000 question-answer pairs from 100 narrative books of Project Gutenberg, LumberChunker reached a Recall@20, the share of relevant passages retrieved among the top 20 chunks, of 77.92 %, outperforming the recursive chunking baseline with 74.35 % [3].

However, these improvements in retrieval accuracy come with high operational and computational costs [13]. Run times depend strongly on the hardware and software used, but LLM-based chunking is generally the most expensive form of pre-processing. Because each prompt depends on the boundary returned for the previous passage, the queries cannot be parallelised [3]. The processing time also grows with document size: segmenting *The Count of Monte Cristo* (14,339 paragraphs) took LumberChunker around 27 minutes, compared to 0.6 seconds for recursive splitting [3].

2.4 Shortcomings in current literature and methodological commitments

Existing literature on document segmentation for RAG shows substantial methodological limitations. First, comparative evaluations frequently test chunking strategies without calibrating the baselines to the average chunk length of the LLM splitters [11]. For example, in the evaluation of LumberChunker, the recursive baselines averaged 399 tokens per chunk, while the LLM chunker produced a mean of 334 tokens [3]. Without normalising the baseline lengths, the quality of the boundary placement is confounded with chunk-size effects, since chunk size directly affects retrieval recall and the density of relevant context [11]. Second, benchmark studies also aggregate retrieval and generation metrics without isolating the structural layout of a document as an independent variable [11], [13]. While LLM chunkers demonstrate advantages on long narrative prose [3], their benefits diminish on real-world, structured text, where natural typographical conventions already provide a good indication of boundary placement [11]. To address these gaps and to isolate the effect of chunking, this thesis follows four methodological commitments:

1. **Length-matched baselines.** All baselines are tuned to the mean chunk length produced by the LLM chunker on each document (section 4.2). This removes the chunk-size bias and isolates the quality of the boundary placement [11].
2. **Document structure as an independent variable.** The evaluated documents belong to three structure classes (explicit, implicit and unstructured, section 4.1), in order to evaluate under which structural conditions LLM-based chunking justifies its computational overhead [11], [13].
3. **Local execution and determinism.** To make the project usable for everybody and to work within the constraints of the available hardware, no commercial API is used; all segmentation runs use an open-weight local model with deterministic decoding ($T = 0$ and a fixed seed) [13].
4. **Pre-specified analysis.** The metrics, the paired significance test and the correction for multiple testing were fixed in a written test protocol before the final chunking runs; every decision taken after results were known is reported as such (section 4.9).

Chapter 3

Implementation

3.1 Overview and design goals

The project is deliberately split into two parts: `llm-semantic-chunker`, a library published on PyPI that turns a string into a list of chunks, and everything under `app/` and `eval/`, the evaluation harness that produces the results of chapter 4. The split ensures that a user of the library does not have to install anything besides the chunker itself, and the packaging configuration enforces it. The library has three runtime dependencies: an HTTP client, a sentence tokeniser and a language detector. A vector store, a PDF reader and the two framework adapters for LangChain and LlamaIndex are optional extras.

Hardware constraints. Several design decisions follow from the hardware the project was developed and evaluated on: a MacBook Air with an M1 chip and 8 GB of memory. Three processes have to be active during an entire evaluation run: Ollama running `qwen3.5:4b` for the boundary decisions (about 3.5 GB, largely wired and therefore not swappable), Ollama serving `bge-m3` for the embeddings (about 1.2 GB) and the Python process holding the chunks and the vector store (0.5 GB to 1.5 GB). As a consequence, free memory fell below 60 MB for extended periods and the swap file grew to 15 GB.

Two design decisions follow directly from these constraints. First, embeddings are computed by a second Ollama-served model rather than in-process, so that no PyTorch runtime occupies additional memory in the Python process. Second, chunking and evaluation are split into two phases with a persistent cache between them: chunking a document takes hours on this hardware, whereas evaluating the chunks takes only minutes, and the cache ensures that an interruption does not destroy hours of chunking work. Section 3.5 describes what the cache records and why.

Scope. The library implements chunking only and deliberately leaves out reranking, query expansion, hybrid retrieval and similar components. These belong to the retrieval stage and could raise the hit rates, but the improvement would not come from the chunking and would distort the comparison with the baseline strategies. Whether the effect of LLM-based chunking persists with a stronger retrieval stage is a worthwhile question, but it is outside the scope of this work.

3.2 Architecture

The project is designed so that each stage of the chunking and of its post-processing can be replaced without affecting the others. The LLM client is therefore defined by a structural protocol: any object with a `chat(messages)` method can be used, which removes the hard dependency on Ollama. The prompts are dataclasses that can be substituted, and the post-processors share a common interface, so that they can be used interchangeably.

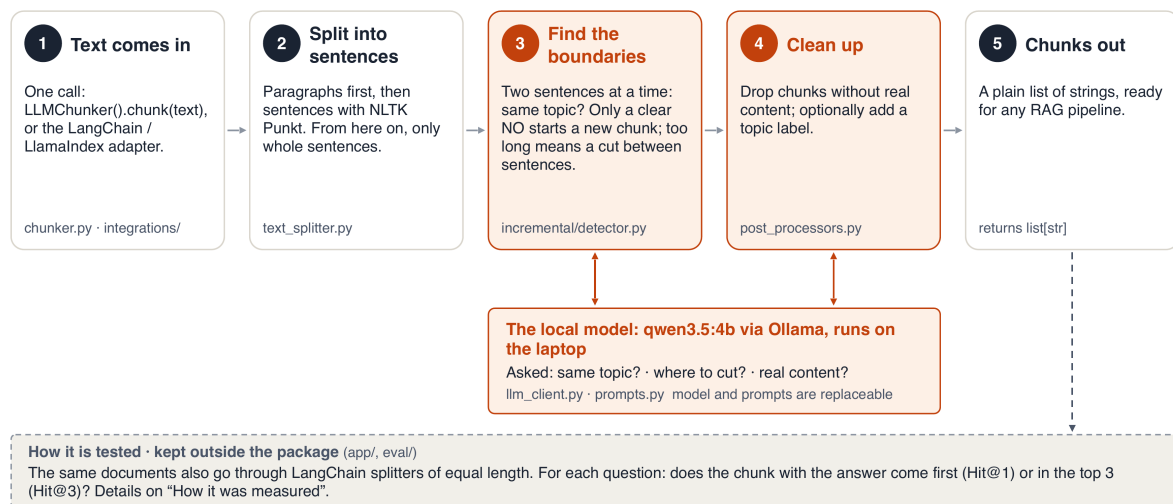


Figure 3.1: Architecture of the library. A document passes five stages; the local model is asked during boundary detection (step 3) and by the optional post-processors (step 4). The evaluation harness is kept outside the package.

3.2.1 Incremental boundary detection

The LLM-based chunking rests on two principles. First, the decision whether a sentence belongs to the current chunk is *incremental*: the model never sees the whole document, only the current chunk and the next few sentences. This keeps every LLM call far below the context window (4096 tokens by default) and makes the cost linear in the document length. Second, merging is implicit: sentences are joined into chunks while the model moves through the document, not in a separate step afterwards. Listing 3.1 shows the core of the loop.

Listing 3.1: The assembly loop, condensed.

```
while pos < len(sentences):
    candidate = sentences[pos:pos + self.step_sentences]
    pos += self.step_sentences

    if not current:                                # first group of a chunk
        current = list(candidate)
    elif self._same_topic(current, candidate, pos): # one LLM call
        current.extend(candidate)
    else:                                           # topic boundary
        chunks.append(" ".join(current))
```

```

current = list(candidate)

while self._over_limit(current):                                # size cap, see below
    idx = self._fit_split(current)
    chunks.append(" ".join(current[:idx]))
    current = current[idx:]

```

The condition for closing a chunk and starting a new one is deliberately conservative. Only an explicit NO from the model sets a boundary; an unclear answer continues the current chunk. The reason is that, because the process is incremental, a wrong split cannot be repaired later, whereas a missed boundary merely produces a longer chunk.

3.2.2 Size cap and split policy

The topic decisions alone put no upper limit on chunk length: a twenty-page document about one subject would in theory become a single, unusable chunk. The settings `max_chunk_sentences` and `max_chunk_chars` therefore force a split once a chunk exceeds either threshold, regardless of the topic.

Two policies decide where this cut is made, and both cut only between sentences. With `smart_split`, the default, the model reads the over-long chunk and chooses the sentence at which the split fits best; the chunker then moves this index backwards until the first piece fits the character cap. With `smart_split` disabled, the oversized chunk is cut at its middle sentence. The two policies form one of the ablations in section 4.6: they differ only in where the cap cuts, not in whether it applies.

The size cap is a target, not a guarantee. Because cuts are only made at sentence boundaries, a single sentence that is longer than `max_chunk_chars` is never split and becomes a chunk that exceeds the cap.

3.2.3 Heading awareness

In addition to the topic decisions, the library can force a boundary at headings, a feature that was added during the evaluation. It implements three ways of finding headings. The default mode `"regex"` applies a fixed pattern for numbered and named headings to the sentences after tokenisation. Its successor, the mode `"lines"`, works on the raw text before tokenisation and catches headings such as 3.2. **Error Handling** that the sentence tokeniser would otherwise break apart. The mode `"hybrid"` additionally asks the model about short sentence groups that the pattern did not flag, which makes it possible to measure the model's contribution to heading detection.

3.2.4 Post-processing

Two optional post-processing steps run over the finished chunk list: the low-information filter and the topic enricher. A third one, a merging step, was considered during the design but became unnecessary, because the incremental mode merges while it moves through the document and never revisits chunks it has already closed. In the legacy mode `"window"`, a separate merging

step caused errors: the pre-split pieces could already contain different topics, and a wrong merge diluted the chunk.

The *low-information filter* asks the model for each chunk whether it contains substantive information. Only an explicit NO removes the chunk; the aim is to keep bare headings, boilerplate lines and similar fragments out of the index. Its effect depends strongly on the document: manuals and table-heavy texts lose more chunks than continuous prose such as a non-fiction book.

The *topic enricher* prefixes each chunk with an LLM-generated line [Topic: ...], so that a chunk that is taken out of its document and loses its context still carries a short descriptive label. Enrichment is disabled by default, whereas the low-information filter is enabled. Their effects are evaluated in sections 4.4 and 4.6.

3.2.5 Configuration and the LLM client

All chunker settings are collected in `ChunkerConfig`, an immutable dataclass that validates itself on construction, so that an invalid setting fails before the chunking starts. The chunker accepts either a configuration object or the individual settings as keyword arguments, but refuses both at once, since it would be ambiguous which one takes precedence. The client, `OllamaClient`, wraps Ollama’s chat endpoint and adds three mechanisms that make the output reproducible and complete.

Deterministic decoding. The temperature is set to zero and the seed is fixed (`temperature=0`, `seed=42`), so that repeated runs produce identical output (section 3.4.1).

A prompt-size guard. Ollama silently truncates a prompt that exceeds `num_ctx`, so the model would judge a chunk of which it sees only a part. The client therefore estimates the size of each prompt before sending it (four characters per token) and refuses a prompt that would exceed 90 % of `num_ctx`, rather than letting a truncated judgement propagate through the chunking. A second check reads the token count that Ollama reports back and logs a warning when the prompt comes close to the limit.

Retries. Server errors (5xx) and transport errors, such as a model reload or a dropped connection, are retried with exponential backoff, while client errors (4xx) are raised at once.

3.3 Developer guide

This section is a guide for developers who want to use the library. It covers the installation, the three ways of calling the chunker (the native API and two framework adapters) and the extension points.

3.3.1 Installation

The library is published on the Python Package Index as `llm-semantic-chunker`¹:

Listing 3.2: Installing the library and its optional extras.

¹<https://pypi.org/project/llm-semantic-chunker/>

```

pip install llm-semantic-chunker
pip install "llm-semantic-chunker[langchain]"      # LangChain adapter
pip install "llm-semantic-chunker[llamaindex]"    # LlamaIndex adapter, Python 3.10+
pip install "llm-semantic-chunker[pdf]"           # PDF text extraction
pip install "llm-semantic-chunker[vectorstore]"   # ChromaDB wrapper

```

The base installation brings only the three runtime dependencies named in section 3.1 and is sufficient to chunk strings; everything beyond that is an extra that is installed on demand.

The chunker is meant to be a small component of a larger pipeline. Computing embeddings in-process with sentence-transformers would have turned an installation of about 40 MB into one of well over 800 MB and tied every project using the library to the PyTorch version used here. Delegating the embeddings to a second Ollama-served model avoids this dependency instead of hiding it behind an optional extra. It also needs less memory, which was decisive given the hardware constraints described in section 3.1.

Chunking also requires a local Ollama chat model. The default is `qwen3.5:4b`, but any other Ollama model can be set in the client. On its first call, the sentence splitter downloads NLTK’s punkt data once; later runs reuse it.

3.3.2 The native API and the framework adapters

Listing 3.3: Chunking a string with the native API.

```

from llm_semantic_chunker import LLMChunker, OllamaClient

chunker = LLMChunker(client=OllamaClient(), max_chunk_chars=1200)
chunks = chunker.chunk(text)          # -> list[str]

```

The return value is a plain list of strings. Character offsets, the governing heading and similar metadata were deliberately left out, because this work is limited to evaluating whether LLM-based chunking sets better boundaries than the baseline strategies.

The frameworks are integrated through adapters, so that an existing splitter can be replaced without changing the rest of a pipeline (listing 3.4). `LLMSemanticSplitter` implements LangChain’s `TextSplitter` interface, so `create_documents()`, `split_documents()` and `transform_documents()` come from the base class. `LLMSemanticNodeParser` does the same for LlamaIndex: it subclasses the `TextSplitter` of LlamaIndex, which is itself a `NodeParser`, so that the method `get_nodes_from_documents()` works directly. In both frameworks, the adapter only implements `split_text()`; everything else is built on top of it.

Listing 3.4: Replacing an existing splitter. Only the constructor line changes.

```

# instead of: RecursiveCharacterTextSplitter(chunk_size=700, chunk_overlap=50)
from llm_semantic_chunker.integrations.langchain import LLMSemanticSplitter
splitter = LLMSemanticSplitter(max_chunk_chars=1200)
docs = splitter.create_documents([text])          # -> list[Document]

# instead of: SentenceSplitter(chunk_size=700, chunk_overlap=50)
from llm_semantic_chunker.integrations.llamaindex import LLMSemanticNodeParser
parser = LLMSemanticNodeParser(max_chunk_chars=1200)

```

```
nodes = parser.get_nodes_from_documents([Document(text=text)]) # -> list[TextNode]
```

The LlamaIndex extra requires Python 3.10 or newer, because `llama-index-core` depends on `banks`, which annotates fields as `Path | None`. This syntax raises a `TypeError` on Python 3.9, which the chunker itself still supports. LlamaIndex is therefore kept in a separate extra.

3.3.3 Extension points

The language model. Any object with a `chat(messages) -> str` method can be passed as `client`, for example a wrapper around a hosted API or another local server.

The prompts. Each kind of model decision has its own prompt class, which can be extended or replaced through a keyword argument: `incremental_prompt` for judging topic shifts, `low_info_prompt` for removing low-information chunks and `enrichment_prompt` for adding a topic line.

Logging. All library messages go through the `logging` module behind a `NullHandler`, so that the application decides what is shown; `verbose=True` only attaches a console handler to the package logger.

What is not replaceable. The assembly loop. It defines the LLM-based strategy itself, and changing it would interfere with the structure of the project.

3.4 Verification

This section describes how the determinism of the library and the validity of the question sets were checked. The scripts are in the repository under `eval/experiments/` and `tools/`.

3.4.1 Determinism

The evaluation relies on the chunking being deterministic. Repeating full chunking runs of the evaluated documents would have taken days, so `run_determinism.py` tests this on a shorter document instead: it chunks the document several times with identical settings and compares a hash over the chunk lists. In three runs, it produced 48 chunks each with a mean length of 673 characters, the identical digest and identical boundary statistics (27 topic decisions, 28 size-cap splits, 8 heading and 9 end boundaries, counted before the low-information filter).

3.4.2 The question sets

The question sets are checked as well: `tools/verify_questions.py` tests every question against its document. The anchor must occur in the document verbatim after whitespace normalisation and exactly once, it must be at least 40 characters long, and no anchor may appear twice in a set. All 1483 anchors of the three sets used in chapter 4 pass, with lengths between 60 and 297 characters.

3.5 The evaluation harness

The evaluation harness is configured by a `ChunkerConfig` and an `EvalConfig`. `Pipeline` takes a document and runs two phases: first chunk and cache, then embed, retrieve and score. `StrategyCollector` assembles the baseline strategies, the LLM chunker and all ablation arms, each of which differs from the reference configuration in exactly one setting. Two safeguards keep the cached chunks consistent. First, each arm’s cache name is derived from its configuration, so that a setting and its cached file cannot drift apart. Second, each cache stores a fingerprint of the code that drew its boundaries, a hash over the syntax trees of the relevant modules, so that results from different code versions cannot be mixed.

3.6 Use of AI tools

Claude Code, with different Claude models, was used in VS Code as a pair programmer for the library and the evaluation harness. It wrote and ran most of the evaluation and analysis scripts and reviewed code and results. It was also used for research for this thesis, to build tables and figures, for the presentation, and to proofread the text of this thesis for spelling, grammar and consistency. Furthermore, the question sets of chapter 4 were generated by an external language model (Claude Sonnet 5) as part of the method (section 4.1.1). The research methods, the choice of documents and the design, architecture and implementation decisions were not delegated to AI tools. Every review finding was checked against the code or measured before it was accepted, and the main results were recomputed by a second, independent implementation (section 4.9.1).

3.7 Availability

The source code is available at <https://github.com/antunoviic/semantic-chunking> under the MIT license, and the library is published on PyPI as `llm-semantic-chunker` (version 1.0.2). Besides the library, the repository contains the evaluation harness, the three evaluated documents, the question sets and the chunk lists of the evaluated arms, so that the evaluation of chapter 4 can be repeated without the hours of chunking.

Chapter 4

Evaluation

This chapter evaluates whether the chunks produced by the library of chapter 3 are retrieved more accurately than those of the baseline strategies. After the datasets (section 4.1) and the design (section 4.2), section 4.3 reports the main comparison; sections 4.4 to 4.8 examine the low-information filter, the source of the differences, the configuration options, parent-child retrieval and the cost, and section 4.9 discusses robustness and validity. Chapter 5 interprets the results.

4.1 Datasets

The evaluation uses three documents, one for each structure class of section 2.4 (table 4.1).

Table 4.1: Evaluation documents. Characters after text extraction, sentences as counted by the library’s sentence splitter, and questions that passed the verification (section 4.1.1).

Document	Source and format	Structure class	Characters	Sentences	Questions
rfc9110	RFC 9110, HTTP Semantics [4]; plain text	explicit	502,906	3882	549
nasa	NASA Systems Engineering Handbook [10]; PDF, 297 pages	implicit	813,744	6071	534
wells	H. G. Wells, <i>A Short History of the World</i> [14]; plain text	unstructured	616,390	4754	400

rfc9110 is an explicit text: a technical standard with numbered sections throughout. **wells** is continuous narrative prose about different historical periods, in which the topic shifts are fluid throughout the document. **nasa** (NASA SP-2016-6105 Rev2) changes topic from paragraph to paragraph, but loses its layout during text extraction. The project’s PDF reader joins the lines of a page into one paragraph (310 paragraphs, median 2830 characters), and the running header appears inside the text on almost every page.

4.1.1 Question sets

The questions were generated by an external language model (Claude Sonnet 5), never by the local model under test. The prompt requires every question to come with an anchor: a passage of 60 to 300 characters, copied verbatim from the document, that contains the answer. Furthermore,

the prompt requires a question to be answerable from one section alone and its anchor to appear only once in the text. An automatic check (section 3.4.2) then tests every candidate against the extracted text: the anchor must be verbatim after normalisation, at least 40 characters long, not duplicated and unique in the document. Candidates that fail are discarded (16 of 550 on `nasa`, 4 of 553 on `rfc9110`). No questions were removed or added by hand afterwards.

4.2 Experimental design

4.2.1 Compared strategies

The LLM chunker is evaluated in two arms that share one configuration: two sentences per decision (`step_sentences=2`), a size cap of 1200 characters, at most 100 sentences per chunk and no heading detection. The *tested arm* also switches off the low-information filter, which the library enables by default, so that it evaluates the chunking alone, without any post-processing. The *filter arm* is the same configuration with the filter; it shows the effect of the filter as an optional add-on (section 4.4) and is the reference for the ablations (section 4.6). Table 4.2 lists the five baselines.

Table 4.2: Baselines. All are length-matched to the LLM chunker (see text).

Strategy	Implementation
Fixed-size	<code>LangChain CharacterTextSplitter</code> , space as separator, overlap of 50 characters.
Recursive (default)	<code>LangChain RecursiveCharacterTextSplitter</code> with its default separator order: paragraph, line break, space. A paragraph longer than the target is cut at the last space that fits, usually inside a sentence. Overlap of 50 characters.
Recursive (sentence ends)	The same class with a sentence end (whitespace after <code>.</code> , <code>!</code> or <code>?</code>) as second separator, before line break and space, so that an over-long paragraph is cut after the last complete sentence that fits. Overlap of 50 characters.
Semantic (tuned)	<code>LangChain SemanticChunker</code> with the same <code>bge-m3</code> embeddings. The percentile threshold is set by bisection to reach the target length, and chunks above 1200 characters are divided at sentence boundaries.
Packing (no model)	The library’s own <code>LLMChunker</code> with a stand-in client that always answers “same topic”, so that every boundary comes from the size cap, which cuts between sentences (at the middle sentence).

Length matching. Because chunk length can influence the hit rates, every baseline is tuned to the mean length of the LLM chunks that the filter keeps (714 characters on `nasa`, 720 on `rfc9110`, 725 on `wells`). The tested arm’s own mean is lower (454, 621 and 711 characters), because it also contains the chunks that the filter removes, mostly short fragments such as 464 chunks of the `nasa` table of contents that consist of nothing but dot leaders. A splitter’s mean length cannot be set directly, so its parameters (chunk size, percentile threshold etc.) are adjusted until the mean lies within 4 % of the target.

4.2.2 Retrieval and hit criterion

Chunks and questions are embedded with bge-m3 [2], served by a local Ollama instance. Retrieval uses exact search: each question is compared with every chunk by cosine similarity, and the chunks are ranked by this score. The approximate HNSW index [8] that the project’s vector store uses by default does not guarantee the true nearest neighbours; in this project, its ranking changed each time the same data was indexed again, so it was not used for the final results.

Each question is judged by how much of its anchor is present in a chunk. A question counts as a hit at rank k (Hit@ k) if one of the k highest-ranked chunks contains at least 80 % of the anchor as one continuous passage. The threshold accepts an anchor whose last words fall into the next chunk, but not one that a chunk boundary splits in half. Line breaks and repeated spaces are ignored, so that formatting differences between document and chunks do not matter, and the passage must be continuous, so that a chunk does not count merely because it shares words with the anchor. Applied to all chunks of a strategy instead of only the top k , the same test gives the share of *anchors intact*: if no chunk contains at least 80 % of the anchor, the chunking has cut the answer apart and no retriever can find it. This share is therefore an upper bound for Hit@ k at every k . Hit@1 and Hit@3 are the reported metrics.

4.2.3 Statistical analysis

All strategies are evaluated on the same questions, so they can be compared question by question. When the tested arm is compared with one baseline, only the questions on which the two disagree are informative: b is the number of questions that only the LLM arm answers, and c the number that only the baseline answers. Questions that both or neither answer are ignored. If both strategies were equally good, each of these disagreements would go to the LLM arm or to the baseline with a probability of 1/2. The exact McNemar test [9] computes the two-sided probability p of a split at least as uneven as the observed one under this assumption. The size of the effect is the difference in hit rate, $d = (b - c)/n$ with n questions, in percentage points (pp), with a 95 % Wald interval for paired proportions, $d \pm 1.96\sqrt{((b + c)/n - d^2)/n}$, which gives the range of differences compatible with the data; an interval that contains zero means that no difference can be established. The intervals are not adjusted for multiple testing.

The tested arm is compared with five baselines on three documents at two values of k , which gives a total of 30 tests. Holm’s procedure [5] controls the family-wise error rate, the probability of at least one false positive among the 30 tests, at $\alpha = 0.05$: the p -values are sorted in ascending order, the i -th smallest is compared with $\alpha/(30 - i + 1)$, and the procedure stops at the first comparison that fails. Equivalently, the i -th smallest p -value is multiplied by $30 - i + 1$, capped at 1 and never allowed to fall below the adjusted value before it; these *adjusted* p -values are reported and can be compared with α directly, whereas *unadjusted* values refer to a single test.

Table 4.3: Hit rates in percent. The tested arm is compared with the five baselines; the filter arm with the low-information filter as an add-on is shown for reference (section 4.4).

Strategy	nasa ($n = 534$)		rfc9110 ($n = 549$)		wells ($n = 400$)	
	Hit@1	Hit@3	Hit@1	Hit@3	Hit@1	Hit@3
LLM (tested arm)	69.5	87.1	74.9	90.2	84.0	92.8
Recursive (sentence ends)	66.9	84.3	71.6	88.0	89.8	96.2
Packing (no model)	64.8	82.6	71.6	87.4	86.5	94.8
Semantic (tuned)	66.5	83.3	73.6	89.8	83.2	93.8
Recursive (default)	61.6	80.3	71.8	88.0	87.2	95.0
Fixed-size	61.2	76.8	58.5	80.5	77.8	89.8
LLM with filter (add-on)	68.0	83.5	73.6	88.3	84.0	92.8

4.3 Results

4.3.1 Retrieval accuracy

Table 4.3 shows a pattern that the individual tests confirm. On **nasa** and **rfc9110**, the tested arm reaches the highest hit rates of the six compared strategies at both k (Hit@1 69.5% and 74.9%), but it leads the sentence-aware baselines by only 0.4 to 4.7 pp. On **wells**, three baselines are ahead of it; the clear winner, the recursive splitter with sentence ends, reaches 89.8% against 84.0%. Fixed-size splitting is the weakest strategy on every document. At $k = 3$, most differences on **rfc9110** and **wells** shrink, because a passage missed at rank 1 is often found at rank 2 or 3.

4.3.2 Paired comparisons

Six of the 30 comparisons remain significant after the Holm correction, all against the two naive splitters. Against fixed-size splitting, the tested arm leads on **rfc9110** by +16.4 pp at Hit@1 [+12.0, +20.8] and +9.7 pp at Hit@3 [+6.2, +13.1], and on **nasa** by +8.2 and +10.3 pp (adjusted $p \leq 0.021$). Against LangChain’s default recursive splitter, it leads on **nasa** by +7.9 pp at Hit@1 [+3.2, +12.5] and +6.7 pp at Hit@3 (adjusted $p = 0.030$ and 0.014). The nearest miss is in the opposite direction: the lead of the recursive splitter with sentence ends on **wells** (−5.8 pp at Hit@1), which is not significant after the Holm correction (adjusted $p = 0.064$).

Figure 4.1 makes the structure of the result visible. Against fixed-size splitting, all four intervals on **nasa** and **rfc9110** lie above zero; against the default recursive splitter, the two on **nasa** do. Against the three baselines that keep sentences intact (recursive with sentence ends, the tuned semantic chunker and packing without the model), the point estimates on **nasa** and **rfc9110** lie between +0.4 and +4.7 pp, and nine of the twelve intervals contain zero. On **wells**, the LLM arm is behind three of the five baselines at Hit@1 and four at Hit@3.

4.4 The low-information filter

The filter is an optional post-processing step: after chunking, the model judges every chunk, and the ones without substantive information are removed. Added to the tested configuration, it drops 761 of 1786 chunks on **nasa** but only 9.8% of the text, mostly fragments such as the dot leaders of the table of contents. It also deletes answers: 27 anchors of **nasa** and 11 of **rfc9110**

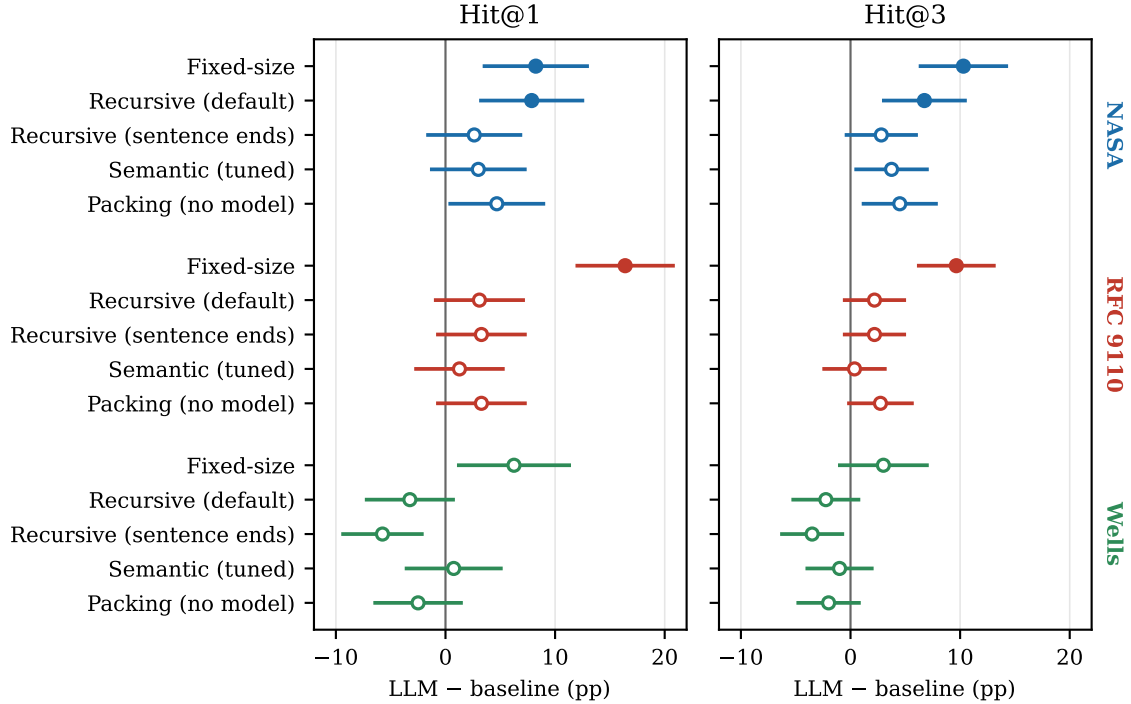


Figure 4.1: Difference in hit rate between the tested LLM arm and each baseline, with 95% intervals. Filled markers are significant after the Holm correction over all 30 tests. Values right of zero favour the LLM chunker.

are no longer contained in any chunk. On *nasa*, 18 of these 27 lie in the reference lists, from which the generating model produced 35 questions although the prompt excluded them. As a result, the filter lowers Hit@3 by 3.6 pp on *nasa* and by 1.8 pp on *rfc9110* (Holm-adjusted $p = 0.002$ and 0.032 over the six tests of this comparison) and leaves *wells* unchanged.

The filter arm, tested against the five baselines with its own Holm family of 30 tests, keeps only two significant results, both against fixed-size splitting on *rfc9110* (+15.1 and +7.8 pp). Its leads over the naive splitters on *nasa* at Hit@1 (+6.4 pp against the default recursive splitter, +6.7 pp against fixed-size splitting) are no longer significant (adjusted $p = 0.28$ and 0.18), mainly because the filter deletes passages that some questions ask about.

4.5 Where the differences come from

The anchors are one or two sentences long, so a chunking that never cuts inside a sentence keeps almost every anchor intact, whatever it does between sentences. Table 4.4 shows the proportion of chunks that end without sentence-final punctuation, a proxy for a cut inside a sentence. Fixed-size splitting ends 93 to 97% of its chunks inside a sentence and leaves only 92 to 94% of the anchors intact. LangChain’s default recursive splitter performs better on *rfc9110* and *wells*, whose paragraphs are short, but on *nasa* it finds no paragraph break within the target length and falls back to the space character, so that 83.7% of its chunks end inside a sentence. This is where the LLM arm is furthest ahead, although the situation is created primarily by the text extraction rather than by the document.

Table 4.4: Chunks ending inside a sentence (mid-sent.) and anchors contained to at least 80 % in some chunk (intact), in percent.

Strategy	nasa		rfc9110		wells	
	mid-sent.	intact	mid-sent.	intact	mid-sent.	intact
LLM (tested arm)	3.1	98.7	11.9	99.3	7.1	95.0
Recursive (sentence ends)	19.3	98.9	35.2	99.6	16.0	99.5
Packing (no model)	3.8	98.1	18.3	98.9	3.4	96.2
Semantic (tuned)	3.4	97.9	6.5	98.9	1.2	95.8
Recursive (default)	83.7	94.6	37.6	99.5	34.6	98.0
Fixed-size	94.8	92.1	93.0	93.1	96.7	93.8
LLM with filter (add-on)	4.0	93.6	9.2	97.3	6.0	95.0

The strongest evidence on the contribution of the model comes from packing, which uses the same sentence splitter and assembly loop without asking the model anything. It trails the tested arm by 4.7 pp on **nasa** and 3.3 pp on **rfc9110** at Hit@1, neither difference significant after the correction, and leads it by 2.5 pp on **wells**. On **wells**, 77 % of the LLM arm’s boundaries are forced by the size cap rather than set by a topic decision (on **nasa** 46 %, on **rfc9110** 53 %), so there the chunker behaves largely like packing with a different split point. The model’s topic decisions produce boundaries that differ from packing, but no retrieval benefit that remains significant after the correction.

4.6 Ablations of the LLM arm

Three options of chapter 3 were evaluated as single-setting changes of the filter arm, the tested arm of the original design, produced in the same chunking run and scored on the same questions. These comparisons are exploratory only; none of the differences would survive a correction for multiple testing (the smallest unadjusted p is 0.029).

Split point at the size cap. Splitting an over-long chunk at its middle sentence instead of at the point chosen by the model (**nasa** only) changes Hit@1 by -0.2 pp and Hit@3 by -2.2 pp ($p = 0.13$).

Heading detection. Neither heading mode (line pattern or hybrid with the model) changes a hit rate by more than 0.7 pp on any document. On **wells**, the line-pattern arm is identical to the reference, and on **nasa**, whose line structure is lost in extraction, the chunk lists differ from the reference by only 7 (line pattern) and 52 (with the model) chunks, so that heading detection has little to change on these two documents.

Topic enrichment. A model-generated topic line in front of each chunk lowers Hit@1 on all three documents (-0.9 , -2.2 and -1.8 pp) and Hit@3 as well. The label therefore does not help the retriever find the passage.

4.7 Parent-child retrieval

The evaluation harness also implements parent-child retrieval, a retrieval technique that is independent of the chunking and is used here as an additional condition under which the strategies are compared. Every chunk (the parent) is divided into children of 250 characters (with LangChain’s recursive splitter and an overlap of 30 characters), the children are embedded and searched, and the parents of the best children are returned, each parent at most once. It was applied to the tested arm and to all five baselines, with the same questions, search and hit criterion as in section 4.3.

Table 4.5: Hit rates in percent with parent-child retrieval, applied to every strategy. In parentheses: change of Hit@1 against the same strategy without parent-child (table 4.3), in pp.

Strategy	nasa		rfc9110		wells	
	Hit@1	Hit@3	Hit@1	Hit@3	Hit@1	Hit@3
LLM (tested arm)	72.5 (+3.0)	88.4	80.0 (+5.1)	91.1	86.2 (+2.2)	92.5
Recursive (sentence ends)	73.0 (+6.2)	87.6	82.9 (+11.3)	92.7	90.0 (+0.2)	97.5
Packing (no model)	73.0 (+8.2)	86.0	76.3 (+4.7)	91.6	85.5 (−1.0)	93.2
Semantic (tuned)	71.2 (+4.7)	88.2	77.6 (+4.0)	90.5	89.0 (+5.8)	94.5
Recursive (default)	64.2 (+2.6)	81.1	82.7 (+10.9)	92.5	89.2 (+2.0)	96.2
Fixed-size	57.9 (−3.4)	73.6	71.8 (+13.3)	83.6	78.8 (+1.0)	89.8

Parent-child raises most hit rates, but not evenly (table 4.5), and the baseline splitters gain the most on average. On **rfc9110**, the rule-based splitters gain the most at Hit@1 (fixed-size +13.3 pp, the two recursive splitters +10.9 and +11.3 pp, against +5.1 pp for the LLM arm). On **nasa**, packing and the splitter with sentence ends gain +8.2 and +6.2 pp against +3.0 pp for the LLM arm. On **wells**, the changes are small except for the semantic chunker (+5.8 pp).

With parent-child applied to every strategy, the balance against the sentence-aware baselines shifts towards the baselines. The recursive splitter with sentence ends combined with parent-child retrieval reaches a higher Hit@1 than the LLM chunker with or without it on all three documents (73.0 %, 82.9 % and 90.0 %), at no cost in model calls. Tested in the same way as in section 4.3, with parent-child on both sides, the LLM arm remains significantly ahead only of the two naive splitters (four tests on **nasa**, two on **rfc9110**) and is significantly behind the recursive splitter with sentence ends on **wells** at Hit@3 (−5.0 pp, adjusted $p = 0.040$).

4.8 Cost

Chunking one document with the tested configuration took around three hours on an 8GB MacBook Air, and with the filter, which adds one model call per chunk, around four hours. The run times are only a rough indication, because free memory at times fell below 100 MB and the system swapped. On **rfc9110**, the run without the filter took longer than the one with it, most likely because other processes were running at the same time. Fixed-size and recursive splitting of a whole document take about one second, and the semantic chunker needs embeddings but no language model. Retrieval costs the same for every strategy, so the price of LLM-based chunking is paid entirely at indexing time.

4.9 Robustness and threats to validity

4.9.1 Robustness

The hit rates of all arms in table 4.3 and the 30 tests of the filter arm were recomputed by a second, independent implementation of the search, the hit criterion and the tests, with identical results. Table 4.6 shows how the result depends on analysis decisions. The absence of a significant lead over the sentence-aware baselines holds in every variant, and so does the lead over fixed-size splitting on `rfc9110`. The lead over the naive splitters on `nasa` is less stable: it disappears when the filter is added, and without the 35 questions from the reference lists, an exclusion decided after seeing the results and therefore reported only as a sensitivity analysis, the lead over the default recursive splitter at Hit@1 is no longer significant, although the one at Hit@3 is.

Table 4.6: Dependence of the result on analysis decisions. The `nasa` columns give the difference at Hit@1 in pp with the Holm-adjusted p in parentheses; the last column counts significant leads of the LLM arm over a sentence-aware baseline.

Variant	Significant	nasa, Hit@1		Sentence-aware
		vs. recursive	vs. fixed-size	
Main analysis (tested arm)	6 of 30	+7.9 (0.030)	+8.2 (0.021)	none
Tested arm without the 35 reference-list questions (post hoc)	5 of 30	+7.4 (0.067)	+8.0 (0.038)	none
Filter arm (original design)	2 of 30	+6.4 (0.279)	+6.7 (0.176)	none
Filter arm without the 35 reference-list questions (post hoc)	6 of 30	+8.2 (0.024)	+8.8 (0.012)	none

4.9.2 Threats to validity

Internal validity. The questions were generated by a language model, and 35 of the `nasa` questions violate the generation rules, but they were kept because the procedure fixed in advance allowed no manual removal. Exact search, the length target, the additional baselines and the size of the test family were decided after the first test runs were known (section 5.2), with the intention of making the comparison fairer. The choice of the filter-free configuration as tested arm was made after the final results were known and raises the number of significant tests from two to six, but both results (with and without the filter) are reported. In addition, the fixed-size and default recursive splitters use an overlap of 50 characters, whereas the other strategies use none. The tested and the filter arm were chunked in two separate runs.

External validity. Each structure class is represented by one document, so class and document cannot be separated. `nasa` is the only PDF, and its extraction favours every sentence-aware strategy there. One chunking model, one embedding model and one parameter setting were evaluated.

Evaluation validity. Hit@ k on verbatim anchors measures whether the passage containing the answer is retrieved intact, not whether an LLM answers correctly based on the retrieved

chunk or whether a chunk reads coherently. A chunking strategy that keeps topics together but splits an anchor passage can therefore be penalised.

Chapter 5

Findings and Lessons Learned

5.1 Summary of findings

The research question asked whether a local language model places better chunk boundaries than rule-based chunking techniques, and under which conditions. The evaluation of chapter 4 answers it.

Better than splitters that cut sentences apart. The LLM chunker retrieves significantly better than fixed-size splitting and LangChain’s default recursive splitter: on `rfc9110` by +16.4 pp at Hit@1 against fixed-size splitting, and on `nasa` by +8.2 and +7.9 pp against the two. These gains mainly stem from the fact that both baselines cut inside sentences. On `wells`, whose short paragraphs the default recursive splitter keeps whole, no difference is significant.

Not better than splitters that keep sentences intact. Against the recursive splitter with sentence ends, the tuned semantic chunker and the same chunker without the model, no significant difference remains. The LLM chunker leads by 0.4 to 4.7 pp on `nasa` and `rfc9110` and is behind in five of six comparisons on `wells`, by up to 5.8 pp. None of these differences is significant, which indicates that once a baseline keeps sentences intact, the decisions of the local model bring no measurable gain.

The optional features do not improve retrieval. The low-information filter lowers Hit@3 by 3.6 pp on `nasa` and 1.8 pp on `rfc9110` (section 4.4). Heading detection changes no hit rate by more than 0.7 pp, topic labels lower Hit@1 on all three documents, and letting the model choose the split point at the size cap makes no significant difference (section 4.6). Parent-child retrieval raises most hit rates, but the rule-based splitters gain more from it than the LLM chunker (section 4.7).

Cost and conclusion. Although run times depend strongly on the hardware, LLM chunking takes orders of magnitude longer than rule-based splitting: hours instead of about a second per document. For retrieval as measured here, a length-matched recursive splitter that prefers sentence ends to spaces reaches comparable hit rates, and combined with parent-child retrieval it reaches a higher Hit@1 than the LLM chunker on all three documents. The cost of LLM-based

chunking is paid only once, at indexing time, since afterwards the chunks only have to be retrieved from the vector store. On the benchmarks performed here, however, the difference in retrieval was not large enough to justify that overhead.

5.2 Lessons learned

The most challenging part of the project was the fair evaluation: building baselines that are really comparable, measuring significance and separating the contribution of the language model from that of the sentence splitting. The following lessons come from that part.

5.2.1 A fair comparison needs more than equal chunk length

At first, only the mean chunk length of the baselines was adjusted to the LLM chunker. Evaluating the first results revealed gaps in the experimental design: LangChain’s semantic chunker had to be tuned before it was a fair opponent, the recursive splitter needed a variant that prefers sentence ends, and the same chunker without the model had to be added to isolate the contribution of the model.

5.2.2 Changes to the plan will happen

The test configuration changed over time. For example, exact search replaced the approximate index, and more baselines and a family of 30 tests were added after an external review. Furthermore, the low-information filter was removed from the tested arm because it removed passages that contained answers.

5.2.3 Cached results ensure clean tracking

Chunking takes hours, so the chunks have to be cached, and each cache needs a clear fingerprint of its configuration and code, so that results from different states are not mixed up.

5.2.4 Working with limited hardware

All runs used a laptop with 8 GB of memory, and chunking one document took two to six hours. Computing the embeddings in Ollama instead of the Python process and caching every arm separately made the runs possible. The run time is one of the reasons why the evaluation covers only three documents.

5.3 Limitations and future work

The evaluation covers one document per structure class, the chunking model is a local model with four billion parameters, and the embedding model and the parameter settings were fixed for the entire project. Furthermore, the benchmark measures whether a verbatim passage is retrieved intact, with questions generated by a language model; it does not measure the quality of generated answers or whether a chunk reads as one coherent unit. The results on **nasa** depend

on an extraction that joins each page into one paragraph, and the ablations were run only with the low-information filter.

Future work can address these limitations with several documents per structure class, a larger chunking model, an end-to-end evaluation of the generated answers, a repetition on **nasa** with a layout-preserving extraction and ablations without the filter. For the library, returning the offsets of each chunk and the reason for each boundary (topic decision, size cap or heading) instead of plain strings would make its decisions easier to inspect in an application.

Bibliography

- [1] S. Bennani and C. Moslonka, “A systematic analysis of chunking strategies for reliable question answering,” in *Advances in Information Retrieval: 48th European Conference on Information Retrieval, ECIR 2026, Delft, The Netherlands, Proceedings, Part IV*, R. Campos et al., Eds., ser. Lecture Notes in Computer Science, vol. 16486, Springer, 2026. DOI: 10.1007/978-3-032-21321-1_9
- [2] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “BGE M3-Embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” *arXiv preprint arXiv:2402.03216*, 2024.
- [3] A. V. Duarte, J. D. S. Marques, M. Graça, M. Freire, L. Li, and A. L. Oliveira, “Lumber-Chunker: Long-form narrative document segmentation,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 6473–6486.
- [4] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics,” RFC Editor, RFC 9110, 2022. DOI: 10.17487/RFC9110
- [5] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [6] C. Kiss, M. Nagy, and P. Szilágyi, “Max–Min semantic chunking of documents for RAG application,” *Discover Computing*, vol. 28, no. 1, p. 117, 2025.
- [7] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [8] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [9] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [10] *NASA Systems Engineering Handbook*, NASA SP-2016-6105 Rev2, National Aeronautics and Space Administration, Washington, DC, 2016.
- [11] R. Qu, R. Tu, and F. Bao, “Is semantic chunking worth the computational cost?” In *Findings of the Association for Computational Linguistics: NAACL 2025*, L. Chiruzzo, A. Ritter, and L. Wang, Eds., Albuquerque, New Mexico: Association for Computational Linguistics, Jun. 2025, pp. 2155–2177. DOI: 10.18653/v1/2025.findings-naacl.114

- [12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [13] M. Śmigielski et al., “Chunking methods on retrieval-augmented generation: Effectiveness evaluation against computational cost and limitations,” *Procedia Computer Science (KES 2026)*, 2026.
- [14] H. G. Wells, *A Short History of the World*. 1922, Plain text from Project Gutenberg.